

PROIECT DE DEZVOLTARE SOFTWARE

Sistem de Rezervări pentru Cinema

Documentație Tehnică de Arhitectură și Implementare

Tehnologie: C++ / CMake / NCurses

Paradigmă: Programare Orientată pe Obiecte (POO)

Scop: Gestiune Spectacole, Clienți, Vânzări și Rapoarte Analitice

Data: Mai 2026

1. Introducere și Obiective

Aplicația **Sistem de Rezervări pentru Cinema** reprezintă o soluție software completă bazată pe o consolă interactivă avansată, destinată managementului operațional al unui cinematograful modern. Dezvoltat în limbajul C++, proiectul utilizează paradigme avansate ale Programării Orientate pe Obiecte (POO) pentru a asigura un cod modular, extensibil și performant.

Sistemul consolidează fluxurile esențiale dintr-un cinematograful într-o singură aplicație autonomă:

- **Managementul Portofoliului de Filme:** Înregistrarea producțiilor cinematografice, stocarea metadatelor asociate (durată, gen, rating) și categorisirea lor.
- **Gestiunea Sălilor de Proiecție:** Configurarea capacităților fizice și a tehnologiilor de redare (2D, 3D, IMAX, VIP, 4DX, DolbyAtmos).
- **Planificarea Spectacolelor:** Corelarea filmelor cu sălile disponibile, setarea orelor de difuzare, datelor calendaristice și a prețurilor de bază.
- **Evidența Clienților și Politici Sociale:** Categorisirea clienților pentru aplicarea automată a discounturilor comerciale corelate cu statutul lor social.
- **Procesarea Rezervărilor pe Hartă Interactivă:** Achiziționarea locurilor prin selectare vizuală în timp real direct din matricea sălii.
- **Business Intelligence și Analitice:** Modul dedicat calculului încasărilor totale, determinării indicatorilor de popularitate și fidelitate.

2. Arhitectura Tehnologică și Mediu de Lucru

Aplicația a fost proiectată urmând principii riguroase de inginerie software pentru a minimiza dependențele externe, oferind simultan o experiență de utilizare calitativă printr-o interfață grafică text formatată.

Componentă Tehnologică	Rol și Detalii de Implementare
Limbaaj și Standard	C++ modern. Utilizarea standardului asigură structuri de date sigure (<code>std::vector</code> , <code>std::map</code> , <code>std::string</code>) și gestionare robustă prin fluxuri și pointeri de obiecte.
Interfață Utilizator	Biblioteca NCurses / Curses . Aceasta permite controlul direct al terminalului, randarea de ferestre (windows), cadre decorative, manipularea culorilor de fundal/text și citirea tastelor direcționale pentru navigare interactivă.
Sistem de Compilare	CMake (versiune minimă 3.10) . Oferă o configurare cross-platform curată, localizând automat pachetele necesare pentru legarea (linking-ul) bibliotecii Curses.
Persistența Datelor	Mecanism custom de serializare în fișiere text (<code>date.txt</code>). Structura folosește etichetare pe blocuri de date și parsare bazată pe identificatori (ID-uri) pentru a simula legăturile de tip cheie externă din bazele de date relaționale.

3. Structura Modelului de Date (Clase Core)

Logica de business este mapată pe o ierarhie clară de clase, unde entitățile operaționale colaborează prin pointeri pentru a preveni duplicarea informației în memorie.

3.1. Clasa Film

Încapsulează atributele descriptive ale unei producții cinematografice.

- **Atribute private:** `titlu` (string), `gen` (string), `durata` (int - exprimată în minute), `rating` (float).
- **Metode cheie:** Constructori dedicați, metode de tip getter/setter pentru fiecare atribut și metoda `afisare()` responsabilă de listarea standard în fluxul de ieșire.

3.2. Clasa Sala

Definește parametrii fizici ai spațiului de proiecție.

- **Atribute private:** `numarSala` (int), `capacitate` (int), `tip` (string).
- **Sistemul de categorii:** Suportă mapări flexibile, permițând tarife diferențiate la nivelul spectacolului în funcție de tehnologia sălii (de ex. standard 2D, IMAX premium sau scaune dinamice 4DX).

3.3. Clasa Client

Stochează identitatea și profilul utilizatorului, element critic în stabilirea prețului final.

- **Atribute private:** `id` (int), `nume` (string), `prenume` (string), `email` (string), `telefon` (string), `tipClient` (string).
- **Reguli comerciale:** Tipul de client determină politica de reduceri. Sistemul recunoaște și validează patru categorii distincte prezentate în tabelul de mai jos.

Categorie Client	Procent Reducere	Destinație / Condiție Socială
Normal	0% (Fără reducere)	Tarif standard integral aplicabil publicului larg.
Membru fidel	10% Discount	Utilizatori înregistrați în programul de loialitate al cinematografului.
Student	20% Discount	Acces subvenționat pe baza legitimației de studii valabile.
Pensionar	30% Discount	Facilitate socială acordată persoanelor vârstnice.

3.4. Clasa Spectacol

Reprezintă instanțierea unui eveniment programat într-un anumit interval orar.

- **Atribute private:** `id` (int), `film` (Pointer către instanța unică a obiectului Film), `sala` (Pointer către instanța unică a obiectului Sala), `data` (string), `ora` (string), `pret` (float), `locuriOcupate` (`std::vector<bool>`).
- **Mecanismul hărții de locuri:** Vectorul boolean este redimensionat automat la apelul constructorului, utilizând metoda `sala->getCapacitate()`. Valoarea `false` definește un loc disponibil, în timp ce `true` marchează un loc deja rezervat sau vândut.
- **Logica operațională:** Metodele `esteLocLiber(idx)`, `ocupaLoc(idx)` și `elibereazaLoc(idx)` includ mecanisme preventive de validare a bound-urilor vectorului, respingând indexările eronate sau tentativele de dublă ocupare.

3.5. Clasa Rezervare

Leagă clienții de spectacolele alese, încapsulând tranzacția financiară și starea biletelor.

- **Atribute private:** `id` (int), `client` (Pointer la Client), `spectacol` (Pointer la Spectacol), `nrLocuri` (int), `status` (string: "In asteptare", "Confirmata", "Anulata"), `dataRezervare` (string).
- **Polimorfism implicit al costurilor:** Metoda `calculeazaTotal()` interoghează dinamic pointerul de client pentru a-i afla categoria, determină prețul de bază din pointerul spectacolului, înmulțește valoarea cu numărul de locuri și aplică coeficientul de discount corect.

4. Persistența Datelor și Modulul de Stocare

Pentru a evita utilizarea unor sisteme de gestiune a bazelor de date greoaie, proiectul implementează un motor autonom de stocare în clasa `Storage`. Acesta interacționează direct cu fișierul `date.txt`, utilizând fluxuri de citire/scriere (`std::ifstream`, `std::ofstream`) și string streams pentru segmentare.

Formatul fișierului este structurat pe blocuri mari denumite sugestiv, urmate de numărul total de elemente din acea colecție, obiectele individuale fiind separate prin marcajul `---`:

```
[FILME]
10
titlu=Inception
gen=Actiune
durata=148
rating=9.3
---
[SALI]
6
numar=1
capacitate=100
tip=3D
---
[CLIENTI]
...
[SPECTACOLE]
8
id=1
film=0
sala=0
data=2024-03-14
ora=19:00
pret=50
locuri_ocupate=0,1,0,0,0,0,1,0,0,0
---
[REZERVARI]
...
```

Important (Mecanismul de Linkare): La încărcarea datelor, entitățile spectacol și rezervare sunt restaurate prin corelarea indicilor numerici cu poziția obiectelor în vectorii globali de memorie. De exemplu, `film=0` înseamnă că spectacolul va primi adresa primului obiect din vectorul global de filme (`filme[0]`).

5. Interfața Utilizator (Modulul UI bazat pe NCurses)

Aplicația iese în evidență prin utilizarea librăriei **NCurses**, abandonând intrările/ieșirile liniare și monotone de tip `cin/cout` în favoarea unui mediu vizual controlat prin taste.

5.1. Inițializare și Configurare Culori

Funcția `initUI()` preia controlul ecranului, dezactivează afișarea automată a caracterelor tastate (`noecho()`), activează citirea tastelor speciale (`keypad()`) și ascunde cursorul hardware (`curs_set(0)`). Sunt create perechi de culori optimizate pentru lizibilitate:

- `COL_TITLU` (Galben pe Negru) - pentru antete și sigle.
- `COL_SELECTED` (Negru pe Cyan) - pentru elementul activ din meniuri.
- `COL_NORMAL` (Alb pe Negru) - pentru textul informativ curent.
- `COL_SUCCES` (Verde pe Negru) / `COL_ERORARE` (Roșu pe Negru) - mesaje de stare.

5.2. Selectorul de Locuri Interactiv

În cadrul funcției `drawHartaLocuri()`, capacitatea sălii este mapată vizual într-o matrice simbolică (randuri și coloane). Utilizatorul mută un cursor interactiv folosind săgețile de pe tastatură. Locurile libere sunt randate grafic sub forma unor marcaje clare (de ex. `[ ]`), iar cele ocupate sunt colorate diferit și blocate. Selecția multiplă se realizează prin apăsarea tastei *Space* / *Enter*, oferind un feedback vizual instantaneu.

6. Motorul de Rapoarte și Business Intelligence

Clasa `Raport` centralizează logica de analiză financiară și statistică a cinematografului. Aceasta operează exclusiv pe baza rezervărilor care au starea de validare `"Confirmata"`.

Metodele incluse procesează colecțiile prin algoritmi de agregare:

1. **Filmul cel mai popular:** Folosește o structură asociativă `std::map<string, int>` pentru a acumula numărul de locuri vândute per titlu de film, extrăgând la final cheia cu valoarea maximă.
2. **Încasări Totale și Temporale:** Metoda `incasariTotale()` cumulează sumele brute, iar `incasariPeZi(string data)` filtrează încasările izolat pentru o singură zi calendaristică.
3. **Fidelizare Clienți:** Identifică utilizatorul unic care a achiziționat cele mai multe bilete în cadrul sistemului, facilitând campanii de marketing targetate.

Tratarea Excepțiilor: Toate funcțiile de raportare verifică în prealabil dacă există date valide. Dacă vectorul de rezervări este gol sau nicio rezervare nu este confirmată, sistemul interceptează starea și afișează un mesaj de avertizare controlat, prevenind erorile de tip *division by zero* sau accesul la pointeri nuli.

7. Ghid de Compilare și Execuție

Proiectul este configurat cu ajutorul unui script CMake robust (`CMakeLists.txt`). Pentru automatizarea procesului de build pe mașini Linux sau medii WSL (Windows Subsystem for Linux), este inclus scriptul shell `run.sh`.

Pentru a compila și rula manual proiectul, se folosesc următoarele comenzi în terminal:

```
# Creați un director dedicat pentru build
mkdir -p out/build
cd out/build

# Generați fișierele de build utilizând CMake
cmake ../..

# Compilați executabilul
make

# Rulați aplicația
./SistemRezervariCinema
```

Scriptul de automatizare furnizat (`run.sh`) curăță build-urile anterioare prin parametrul `--clean-first` și lansează automat binarul rezultat, facilitând un ciclu rapid de testare în faza de dezvoltare.

8. Gestiunea Memoriei și Concluzii POO

Fiind un sistem dezvoltat în C++ ce utilizează alocare dinamică pentru colecțiile de obiecte (instanțiate cu operatorul `new` în `SistemRezervariCinema.cpp` și încapsulate în vectori de pointeri), aplicația demonstrează o aplicare solidă a conceptelor fundamentale din programarea orientată pe obiecte:

- **Încapsulare Strictă:** Toate datele sensibile din clase sunt private, accesul fiind permis controlat exclusiv prin metode de interfață (Getteri/Setteri).
- **Asociere și Agregare:** Relațiile complexe (un Spectacol conține un Film și o Sală; o Rezervare conține un Client și un Spectacol) sunt implementate curat prin stocarea adreselor de memorie (pointeri), asigurând consistența datelor pe parcursul rulării.
- **Validare și Robustețe:** Utilizarea containerelor standard previne vulnerabilitățile clasice legate de coruperea memoriei (buffer overflow), oferind o bază stabilă pentru o eventuală extindere viitoare (cum ar fi conectarea la o bază de date SQL reală sau migrarea către o interfață grafică nativă).